MNIST Handwritten Digit Classification with a Multi-Layer Perceptron in PyTorch

[Your Name]
[Course Name] — Final Assessment

Submitted: July 2026

1 Objective

In this assessment I build, train and evaluate a neural network that recognises handwritten digits (0–9) from the MNIST dataset. I implemented the whole solution myself in Python and PyTorch, without using ready-made models, to show that I understand the main concepts of the course: data preparation, model definition, loss functions, gradient-based training and evaluation. The model is judged with Accuracy, Precision, Recall and F1-Score.

2 Methodology and Implementation

Data. I downloaded MNIST and loaded the processed tensors. The training set has 60,000 images and the test set 10,000 images, each of size 28×28 pixels. Figure 1 shows the first ten training images with their labels.

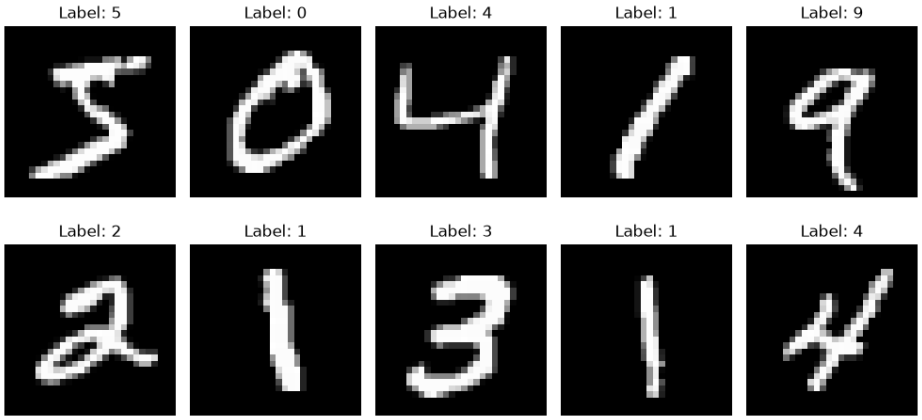


Figure 1: The first ten training digits and their labels.

Preprocessing. I divided all pixel values by 255 so that they lie in $[0, 1]$, which makes training more stable. Each image is flattened into a vector of 784 values before entering the network. I also tested one-hot encoding of the labels to understand the format (for example, digit 5 becomes $[0, 0, 0, 0, 0, 1, 0, 0, 0, 0]$). However, it is not needed in the final code, because PyTorch's `CrossEntropyLoss` accepts the class index directly.

Dataset and DataLoader. I wrote a custom `MNISTDataset` class that inherits from `torch.utils.data.Datas` and implements `__getitem__` and `__len__`. I then used `DataLoader` with a batch size of 64, which worked well on my CPU. The training data is shuffled at every epoch; the test data is not.

Loss and optimiser. I used the cross-entropy loss,

$$\mathcal{L} = -\frac{1}{B}\sum_{i=1}^{B} \log \frac{e^{z_{i,y_i}}}{\sum_{k=0}^{9} e^{z_{i,k}}},$$

where z are the output logits and B the batch size. The weights are updated with stochastic gradient descent (learning rate 0.01) for 20 epochs. As a sanity check, the loss of the untrained network on one batch was 2.308, which is close to $\ln 10 \approx 2.303$, the value expected from random guessing among 10 classes.

Training loop. In each epoch I (1) train on all mini-batches (forward pass, loss, backward pass, optimiser step), then (2) switch to evaluation mode with `torch.no_grad()` and compute the average loss on the test set, which I use as validation loss.

3 Model Architecture

I chose a fully connected network (MLP) with two hidden layers and ReLU activations:

Layer	Type	Output size
Input	Flattened image	784
1	Linear + ReLU	100
2	Linear + ReLU	50
3	Linear (logits)	10

The network has $784 \cdot 100 + 100 + 100 \cdot 50 + 50 + 50 \cdot 10 + 10 = 84{,}060$ trainable parameters. I made the class flexible (the layer widths and activation function are arguments), so other architectures can be tested easily.

4 Training and Validation Results

Figure 2 shows that both the training and the validation loss decrease smoothly during the 20 epochs. The training loss goes from 1.836 to 0.154 and the validation loss from 0.914 to 0.153. Table 1 gives some epochs of the training log.

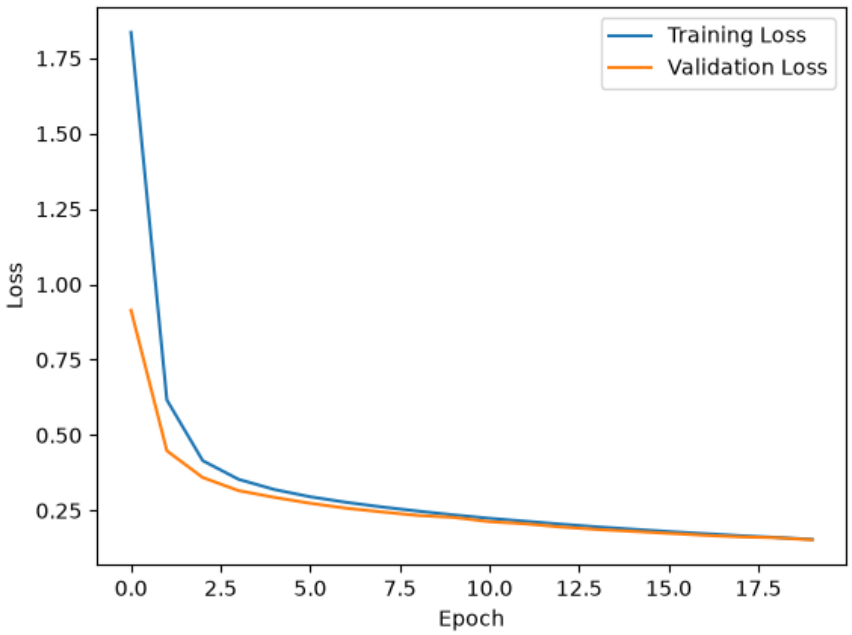


Figure 2: Training and validation loss per epoch.

Epoch	Train loss	Validation loss
1	1.8361	0.9143
5	0.3196	0.2936
10	0.2354	0.2272
15	0.1880	0.1809
20	0.1543	0.1529

Table 1: Selected epochs of the training log.

5 Evaluation Metrics

After training, I predicted the class of each of the 10,000 test images with `argmax` on the output logits, and computed the metrics with scikit-learn. Precision, Recall and F1 are macro-averaged, so each of the 10 digits counts equally.

Metric	Test value
Accuracy	0.9551
Precision (macro)	0.9552
Recall (macro)	0.9544
F1-Score (macro)	0.9546

Table 2: Final test-set metrics.

6 Key Observations and Conclusions

- My MLP reaches **95.5% accuracy** on the test set. Precision and recall are very close (about 0.955), so the model does not favour any particular digit.
- The training and validation losses are almost equal at the end (0.154 and 0.153), so I see no overfitting.

- Both curves were still going down at epoch 20. The model is therefore probably undertrained: plain SGD with a small learning rate learns slowly, especially in the first epochs.
- To improve the results, I could train for more epochs, use a larger learning rate or momentum/Adam, or use wider layers. A convolutional network would likely give above 99%.
- One limitation of my work is that I used the test set also as validation set. A cleaner approach is to split a validation set off the training data and use the test set only once, for the final evaluation.

Overall, this assignment helped me understand the full pipeline of a deep learning project in PyTorch, from raw data to evaluation metrics, and shows that even a small MLP can solve MNIST with good accuracy.